

Assessment, Recommendation & News — How the Reasoning Works

A guided overview of how this dashboard turns raw inputs into a per-symbol Assessment, a discrete Recommendation (Buy / Watch / Hold / Sell), and a ranked, reaction-scored News feed. Everything below is drawn directly from the code in this repository — the real field names, thresholds, label sets, and decision order — not generic investing theory.

The mental model in one line. Notes + fundamentals + market context are distilled by an LLM (or a deterministic rules fallback) into a stored **Assessment**; the **Recommendation** card re-presents that assessment and layers on a **market-grounded sentiment** derived from how the stock actually reacted to recent **News**. Fundamentals and notes drive the thesis; technical signals and news modulate timing and conviction.

These three areas share inputs but answer different questions:

Area	Question it answers	Primary engine	Stored / live?
Assessment	"What's my integrated read on this name right now?"	assessment_service.py + llm_client.py	Stored in assessments (history kept)
Recommendation	"What action, how confident, what's the mood?"	inspector_service.py (build_symbol_recommendation)	Live view over the latest assessment + news sentiment
News	"What moved, how much did it matter, which way?"	news_relevance_service.py + fundamentals_service.py	Live, scored on each fetch (cached)

Personal notes	Price thresholds /	Recent news headlines
(Synthesize -> note synthesis)	targets / buy-below / sell-above + alerts	(yfinance finnhub)
		v
		+-----+
		NEWS RELEVANCE
		daily event study ->
		relevance 0-100 + direction
		(news_relevance_service.py)
		+-----+
v	v	v
+-----+		relevance-weighted
1. ASSESSMENT	assessment_service._build_context	symbol sentiment
+ llm_client.generate_assessment (LLM/rules)		
-> action, confidence, rationale, factors		
stored in `assessments` (+ track record)		
+-----+		
	latest stored assessment	
	v	v
+-----+		
2. RECOMMENDATION	inspector_service.build_symbol_recommendation	
action/confidence/rationale (from assessment) + headline		
+ sentiment (NEWS preferred, notes fallback) + drivers + watch items		
+-----+		
	v	
+-----+		
3. UI	dashboard.html	
Recommendation card / Screening chips : "Assessed" date .		
Summary news feed (relevance badge + reaction) . sentiment chip		
+-----+		

Section 1 — Assessment

Where: `services/assessment_service.py` (orchestration + persistence) and `services/llm_client.py` (the actual wording / scoring).

What an "assessment" is

An assessment is a **stored snapshot** of the app's integrated opinion on one symbol at one point in time. Each row carries a discrete `action`, a `confidence` level, a free-text `rationale`, a list of bullet `factors`, the `note_synthesis` it was built from, the `provider` that wrote it, and a `created_at` timestamp (surfaced in the UI as **"Assessed"**). History is kept but capped — see **Storage & lifecycle** below.

What feeds it (the context bundle)

`AssessmentService._build_context` assembles everything the writer sees (`_build_context`, lines ~229–261). For one symbol it gathers:

Context field	Source	Meaning
<code>currentPrice</code> , <code>targetPrice</code> , <code>analystTargetly</code>	<code>portfolio_service.get_symbol</code>	Price vs. personal & analyst targets
<code>buyBelow</code> , <code>sellAbove</code>	symbol record	Your planned-trade thresholds
<code>noteSyntheseses</code> , <code>unsynthesizedNoteCount</code>	notes on the symbol	Your structured note guidance (and how many notes still need synthesizing)
<code>alerts</code>	<code>alerts_service.list_alerts(status="active")</code>	Active threshold / Fib / screener alerts
<code>fibLevels</code>	<code>fib_service.get_levels</code>	Fibonacci retracement levels
<code>screening</code>	<code>screening_service._score_symbol</code>	Screening <code>score</code> , <code>upsidePct</code> , <code>flags</code> , <code>fibDistancePct</code>
<code>fundamentals</code>	<code>fundamentals_service.get_enrichment</code>	Valuation, growth/profitability, financial health, 52-week range
<code>recentNews</code>	<code>fundamentals_service.get_enrichment</code>	Recent headlines (see Section 3)
<code>holding</code>	<code>holdings_service</code>	Position + portfolio <code>weightPct</code>
<code>technical</code>	<code>technical_signals_service.get_signals</code>	Multi-timeframe trend, momentum, swing/Fib, patterns , and the Confluence block (gated by <code>ASSESSMENT_TECHNICALS</code> , default on)

The `technical` block prefers an **imported** hand-anchored swing/Fib snapshot over the computed one unless the user opted into "prefer computed trends" (`_build_technical`, lines ~263–291).

Who writes it — LLM vs. rules

`LLMClient.active_provider()` picks the writer based on `ASSESSMENT_MODE` and which API keys are present:

Mode / condition	Provider used
<code>ASSESSMENT_MODE=rules</code>	<code>rules</code> (deterministic, offline)
<code>ASSESSMENT_MODE=openai</code> and <code>OPENAI_API_KEY</code> set	<code>openai</code> (<code>gpt-4o-mini</code> default)
<code>ASSESSMENT_MODE=gemini</code> and <code>GEMINI_API_KEY</code> set	<code>gemini</code> (<code>gemini-2.5-flash</code> default)

<code>auto</code> (default)	OpenAI if its key is set, else Gemini if its key is set, else <code>rules</code>
-----------------------------	--

Crucially, the LLM never fails the request. Any LLM error (quota, network, SSL, bad JSON) is caught and falls through to `_rule_based_assessment`, with `llmFallback=True` and `llmError` recorded (`generate_assessment` / `_fallback_assessment`, lines ~119-165). Both paths are normalized through `_normalize_assessment`, which **clamps** the output to the valid label sets.

How it's scored / worded

- **Valid actions:** `buy` | `sell` | `hold` | `watch` (`VALID_ACTIONS`). Anything else normalizes to `hold`.
- **Valid confidence:** `high` | `medium` | `low` (`VALID_CONFIDENCE`). Anything else normalizes to `medium`.
- **rationale:** 2-4 sentences integrating fundamentals, notes, and market context. Empty → `"No rationale provided."`
- **factors:** array of short strings, each meant to cite the **specific** input that drove it (e.g. **"Forward P/E 18 vs 22% revenue growth — reasonable"**).

The LLM prompt's reasoning rules (`_assessment_system_prompt`, lines ~563-608) tell the model to: weigh valuation against growth (a high forward P/E only matters if growth/margins don't support it); treat rising debt-to-equity + weak free cash flow as a risk flag; temper upside when price is far above the 200-day average or near the 52-week high; **lead the technical reasoning from the Confluence block** when present; treat each chart pattern as **one** probabilistic input weighted by its `confidence`, `status`, and volume-Risk `validation` verdict (a `veto/stale` pattern carries little weight); treat news as **sentiment signals, not facts**, and **never fabricate news**. The closing instruction: **technical signals modulate timing and confidence; fundamentals and notes drive the core thesis**.

The rules fallback (`_rule_based_assessment`, lines ~343-420) is a transparent decision ladder. Its action is decided by the first matching threshold trigger, then nudged by note sentiment / target upside:

Trigger (checked in order)	Action	Confidence
<code>sell_above</code> alert active, or <code>price >= sellAbove</code>	<code>sell</code>	high
<code>buy_below</code> alert active, or <code>price <= buyBelow</code>	<code>buy</code>	high
<code>fib_proximity</code> alert active	<code>watch</code>	medium
<code>screener_upside</code> alert active	<code>watch</code>	medium
(else) note synthesis sentiment is <code>bullish</code> and action still <code>hold</code>	<code>watch</code>	medium
(else) personal <code>target</code> implies > 30% upside and action still <code>hold</code>	<code>watch</code>	medium
(no trigger)	<code>hold</code>	medium

It then appends human-readable `factors` from fundamentals (`_fundamentals_factors`) and technicals (`_technical_factors`), e.g. **"Rich valuation: forward P/E 32 vs 9% revenue growth"** (forward P/E > 30 **and** growth < 15%), **"Balance-sheet risk: debt/equity 180 with negative free cash flow"** (D/E > 150 **and** FCF < 0), **"Trading near 52-week high (92% of range)"** (≥ 90% of range), and a Confluence-led technical line.

When / how it's (re)generated

Assessments are **on-demand, never scheduled**. They are produced only when the user triggers:

- `POST /api/v1/assess` → `assess_portfolio` (all symbols, or a passed subset).
- `POST /api/v1/symbols/<symbol>/assess` → `assess_symbol` (one symbol).

For a full portfolio, the slow per-symbol LLM calls run **concurrently** (`ASSESS_WORKERS`, default 6) and are then **persisted serially** so SQLite writes stay ordered (`assess_portfolio`, lines ~95-138). Note **synthesis** (turning a raw note into structured guidance) is a separate, also-on-demand step (`/notes/synthesize`); the assessment consumes the **already-stored** syntheses rather than re-parsing raw notes.

The "Assessed" timestamp

`created_at` on the row is set by the database at insert time. It is returned as `createdAt` (and as `assessedAt` in overview/recommendation payloads) and rendered in the UI as the day portion (`YYYY-MM-DD`) with the full timestamp on hover.

Storage & lifecycle (stored fields)

On save (`_save_assessment`, lines ~305–365) the row is written to the `assessments` table with: `user_id`, `symbol`, `action`, `confidence`, `rationale`, `factors (JSON)`, `note_synthesis (JSON)`, `trading_recommendation (currently always NULL)`, `provider`.

Three side effects fire in the same transaction:

1. **Changelog** — `_record_recommendation_change` logs a row in `recommendation_changelog` **only when the discrete action differs** from the previous assessment (the first-ever assessment for a symbol is skipped). This powers the Summary "recommendation changes" feed.
2. **Track record** — when `TRACK_RECORD` is on (default), `_capture_signal_outcomes` snapshots the recommendation, any non-vetoed/non-stale detected pattern, and a directional Confluence bias as forward-looking "bets" with an entry price and a `TRACK_RECORD_HORIZON_DAYS` (default **21**) evaluation horizon. At most one **pending** capture per (`symbol`, `kind`, `label`) to avoid flooding.
3. **Trim** — `_trim_assessment_history` keeps only the newest `MAX_ASSESSMENTS_PER_SYMBOL = 3` rows per symbol.

Section 2 — Recommendation

Where: `services/inspector_service.py` (`build_symbol_recommendation`, lines ~557–642), assembled per row by `screening_service.run_screen`.

What it is

The Recommendation is **not a second model run** — it is a **presentation layer** over the latest stored assessment, enriched with a market-grounded sentiment and some "what to watch" context. Its `action`, `confidence`, and `rationale` are taken **verbatim from the latest assessment** (lines ~597–603); if no assessment exists yet, it defaults to `hold` / `medium` with a "synthesize then assess" prompt.

Label set

The action labels are exactly the assessment's: **Buy / Watch / Hold / Sell** (the UI capitalizes them; the stored values are lowercase). Confidence is **High / Medium / Low**.

The headline string

`_headline_for_action(action, sentiment)` (lines ~472–484) maps the action to a plain-English headline, optionally suffixed by sentiment:

Action	Base headline
<code>buy</code>	"Consider adding on confirmed setup"
<code>sell</code>	"Consider taking profits or reducing"
<code>watch</code>	"Monitor — catalysts approaching"
<code>hold</code>	"Maintain current positioning"

If `sentiment == "bullish"` and the action is `hold/watch`, it appends "• **bullish growth thesis**"; if `sentiment == "bearish"`, it appends "• **bearish notes flagged**".

Sentiment — the one piece the recommendation adds

This is the key extra computation. Sentiment is chosen by a strict preference order (lines ~578–591):

Priority	Condition	<code>sentiment</code>	<code>sentimentSource</code>	Detail string
1	Qualifying news sentiment exists (<code>news_sentiment.count > 0</code>)	from news	<code>news</code>	the news detail (e.g. "News: 3↑ / 1↓ across 4 recent articles · net +0.42 · top relevance 78")
2	else note-synthesis sentiment ≠ neutral	from notes	<code>notes</code>	"From your synthesized notes"
3	else	<code>neutral</code>	<code>none</code>	"No relevant news or note sentiment — neutral"

The news sentiment is computed by `news_relevance_service.aggregate_symbol_sentiment` (Section 3) and passed in by `screening_service` via a single bulk news fetch (`_news_sentiment_map`). It is **best-effort**: any failure returns `{}` and the recommendation silently falls back to note sentiment.

How it relates to targets / buy-below / sell-above / valuation / technical stance

These do **not** re-decide the recommendation action at the recommendation layer — they already shaped the underlying assessment (the LLM saw them all in context; the rules ladder keys directly off `buyBelow/sellAbove/upside`). The recommendation layer instead surfaces them as **context**:

- `drivers` — cleaned `factors` from the assessment (identifier-looking strings filtered out by `_clean_factors`), capped at 6.
- `watchItems` — upcoming catalysts from note synthesis, active alert messages, and a "Price near Fib (%)" line when within proximity; capped at 8.
- `upsidePct` — passed through from the screening row (computed from analyst or personal target).
- The **technical stance** (the Confluence bias) lives in its **own** Tech Stance column, explicitly described in the UI as **"independent of the Recommendation"** — it is not merged into the recommendation chips.

Confidence & rationale strings

`confidence` is passed straight from the assessment. `rationale` is the assessment's rationale text (or the "synthesize then assess" placeholder). No separate confidence math happens at the recommendation layer.

Section 3 — News

Where: fetching in `services/fundamentals_service.py`; relevance/reaction scoring + sentiment in `services/news_relevance_service.py`; wiring in `api/v1.py` (`/news-feed`, `/news-relevance/<symbol>`); rendering in `dashboard.html`.

Fetching

`FundamentalsService.fetch_recent_news` (lines ~563-582) returns up to `ASSESSMENT_NEWS_LIMIT` (default **6**) normalized headlines per symbol. Provider selection (`active_news_provider`):

Provider	When	Notes
<code>finnhub</code>	<code>NEWS_PROVIDER=finnhub</code> and <code>FINNHUB_API_KEY</code> set	true per-ticker company news, last <code>FINNHUB_NEWS_DAYS</code> (default 30) days
<code>yfinance</code>	default / fallback	<code>Ticker.news</code> , no key required

Both are normalized to a common shape: `{title, publisher, published, link, summary}`. Results are cached (`NEWS_CACHE_TTL_SECONDS`, default 3600s) and a cooldown suppresses repeated failing fetches. If finnhub fails it falls back to yfinance.

Relevance = the daily event study

The core idea (`news_relevance_service.py` module docstring): **relevance is how strongly the market actually reacted to an article**, not keyword matching. For each article, on its first trading day on/after publication (`_event_position`), `_SymbolModel.score` (lines ~184-241) computes:

1. **Raw reaction** — the stock's return that day: `r = close[t]/close[t-1] - 1`.
2. **Abnormal return** — strip out the market: `abnormal = r - beta * index_return`, where `beta` is the stock vs. `NEWS_RELEVANCE_INDEX` (default **SPY**), clamped to `[0, 3]` (`_beta`).
3. **Standardize** — divide by the stock's own normal daily volatility (rolling `NEWS_RELEVANCE_VOL_LOOKBACK`, default **30** days): `z = abnormal / sigma`. So a 3% move on a calm name outranks 3% on a jumpy one.
4. **Volume confirmation** — `_vol_multiplier` scales the score by current vs. average volume, **clamped to `[0.7, 1.25]`** (`0.6 + 0.4 * ratio`).
5. **Squash to 0-100** — `magnitude = tanh(|z| / Z_REF)` (`NEWS_RELEVANCE_Z_REF`, default **2.0**), so a $\sim 2\sigma$ move saturates toward the max), then `relevanceScore = round(min(100, 100 * magnitude * vol_multiplier))`.

It also records `reactionPct` (raw % move), `abnormalPct`, `sigma` (the z), the `volumeRatio`, and a `direction` of `up` / `down` / `flat` from the sign of the abnormal return.

Relevance × recency ranking

The Summary feed (`/news-feed` → `score_and_rank`, lines ~251-311) does **not** sort by pure recency. Each scored item gets a blended rank:

```
_rank = relevanceBase * (0.25 + 0.75 * recency)
recency = 0.5 ^ (age_days / RECENCY_HALFLIFE_DAYS)    # half-life default 5 days
```

So a strong reaction stays near the top even as it ages, but fresh items get a boost. **Unscored** items (no usable price data) get `_rank = -1` and sort after all scored items, ordered by recency. This stops a few heavily-covered names from crowding out genuinely market-moving news.

Phase 2 — on-demand intraday deep-dive

The Fundamentals "Analyze" action calls `/news-relevance/<symbol>` → `score_symbol_intraday` (lines ~610-678). For articles within `NEWS_RELEVANCE_INTRADAY_MAX_AGE_DAYS` (default **10**) it measures the **30-minute** post-publication reaction (`REACTION_WINDOW_MINUTES`, default 30) on 1-minute bars, tagged `reactionWindow="30m"`; older articles fall back to the Phase 1 **daily** score tagged `reactionWindow="1d"`; if even that fails, null scores. Same event-study shape, finer resolution.

Sentiment derivation & surfacing

`aggregate_symbol_sentiment` (lines ~326-412) turns the scored articles for a symbol into one **relevance-weighted directional** sentiment:

- Each article votes `+1` (up) / `-1` (down) / `0` (flat), **weighted by its relevance score**: `net =  $\sum(w \cdot \text{sign}) / \sum w$` , with `w = relevanceScore`.
- **Materiality gate**: the symbol stays `neutral` unless at least one article scores $\geq \text{NEWS_SENTIMENT_MIN_RELEVANCE}$ (default **25**).
- **Label band**: `net  $\geq \text{NEWS\_SENTIMENT\_BAND}$` (default **0.20**) → `bullish`; `net  $\leq -0.20$` → `bearish`; otherwise `neutral`.
- Output: `{sentiment, net, bull, bear, count, topRelevance, detail}`, where the `detail` string is the human-readable sourcing line surfaced on the sentiment chip (e.g. **"News: 3↑ / 1↓ across 4 recent articles · net +0.42 · top relevance 78"**).

This is exactly what the Recommendation prefers for its sentiment chip (Section 2).

Note — a second, unrelated sentiment path. `engine.analyze_asset_sentiment` wraps a transformers sentiment pipeline over arbitrary text. It is **not** part of the news-relevance / recommendation flow documented here; the market-grounded relevance sentiment above is what drives the UI.

How it all surfaces in the UI

Where: `dashboard.html`.

Recommendation card & Screening chips

`renderRecommendationChipRow` (lines ~5188–5205) renders three chips: **ACTION** (`buy/watch/hold/sell`) · **CONFIDENCE** (`high/medium/low`) · **SENTIMENT** (`bullish/neutral/bearish`, colored, with the sourcing detail on hover). The full card (`renderRecommendationCard`) adds the headline, rationale, the `drivers` list, a "What to watch" list, and a meta footer: `Assessed` `<date>` · `<provider>` · `<upside>% upside to target`. The Screening table shows the same chips per row and a separate **"Assessed"** date cell (`renderScreeningAssessedCell`, lines ~5237–5241).

UI help vs. code — reconciled. The Screening help text labels the chips **"ACTION (Buy/Watch/Hold/Sell) · CONFIDENCE (High/Medium/Low) · SENTIMENT (relevance-weighted reaction of recent news; falls back to your synthesized notes)"**. That matches the code exactly: the sentiment source order is news → notes → neutral. The help text capitalizes the labels for readability; the stored/raw values are lowercase.

News feed (Summary) & relevance badge

`renderNewsReactBadge` (lines ~4685–4698) renders the relevance badge: the `relevanceScore` in bold, a direction arrow (▲ up / ▼ down / • flat), the absolute `reactionPct`, and `·No`; the hover title spells out **"Relevance N/100 — market-adjusted price reaction on the news day (... ±X% · Yσ), volume Zx normal"**. Articles with no price data show a plain `—` badge. The feed (`renderTopNews`) defaults to the backend's **relevance x recency** order and also offers **Symbol** and **Time** sort modes plus a **"1 per symbol"** grouping toggle. In Fundamentals, after "Analyze", each article additionally shows a `reactionWindow` tag (`30m / 1d`).

Key knobs at a glance

Knob (env var / constant)	Default	Where	Controls
<code>ASSESSMENT_MODE</code>	<code>auto</code>	<code>llm_client.py</code>	LLM provider selection (auto/openai/gemini/rules)
<code>GEMINI_MODEL</code> / <code>OPENAI_MODEL</code>	<code>gemini-2.5-flash</code> / <code>gpt-4o-mini</code>	<code>llm_client.py</code>	Model used per provider
<code>VALID_ACTIONS</code>	<code>buy/sell/hold/watch</code>	<code>llm_client.py</code>	Allowed action labels (clamped)
<code>VALID_CONFIDENCE</code>	<code>high/medium/low</code>	<code>llm_client.py</code>	Allowed confidence labels (clamped)
<code>ASSESSMENT_TECHNICALS</code>	<code>on</code>	<code>assessment_service.py</code>	Feed computed technicals + Confluence into context
<code>ASSESS_WORKERS</code>	<code>6</code>	<code>assessment_service.py</code>	Parallel LLM calls for portfolio assess
<code>MAX_ASSESSMENTS_PER_SYMBOL</code>	<code>3</code>	<code>assessment_service.py</code>	History retained per symbol
<code>TRACK_RECORD</code> / <code>TRACK_RECORD_HORIZON_DAYS</code>	<code>on</code> / <code>21</code>	<code>assessment_service.py</code>	Capture & later-score signal outcomes
Rules: target-upside watch trigger	<code>> 30%</code>	<code>llm_client._rule_based_assessment</code>	Fallback "watch" on big upside
Rules: rich/cheap valuation	<code>P/E > 30 & growth < 15%</code> / <code>P/E < 20 & growth > 15%</code>	<code>llm_client._fundamentals_factors</code>	Valuation factor wording

Rules: balance-sheet risk	D/E > 150 & FCF < 0	llm_client._fundamentals_factors	Risk factor wording
NEWS_PROVIDER / FINNHUB_NEWS_DAYS	yfinance / 30	fundamentals_service.py	News source + finnhub window
ASSESSMENT_NEWS_LIMIT	6	fundamentals_service.py	Headlines fetched per symbol
NEWS_CACHE_TTL_SECONDS	3600	fundamentals_service.py	News fetch cache TTL
NEWS_RELEVANCE_INDEX	SPY	news_relevance_service.py	Market benchmark for abnormal return
NEWS_RELEVANCE_HISTORY_PERIOD	6mo	news_relevance_service.py	Daily history window for scoring
NEWS_RELEVANCE_VOL_LOOKBACK	30	news_relevance_service.py	Volatility / avg-volume lookback
NEWS_RELEVANCE_Z_REF	2.0	news_relevance_service.py	σ that saturates magnitude
NEWS_RELEVANCE_HALFLIFE_DAYS	5	news_relevance_service.py	Recency half-life in ranking
Volume multiplier clamp	[0.7, 1.25]	news_relevance_service._vol_multiplier	Volume confirmation bounds
NEWS_SENTIMENT_MIN_RELEVANCE	25	news_relevance_service.py	Materiality gate for sentiment
NEWS_SENTIMENT_BAND	0.20	news_relevance_service.py	$\backslash net\backslash$ band that flips off neutral
REACTION_WINDOW_MINUTES	30	news_relevance_service.py	Intraday reaction window
NEWS_RELEVANCE_INTRADAY_MAX_AGE_DAYS	105	news_relevance_service.py	Max article age for intraday tier

One-paragraph recap

When you press **Assess**, the app bundles your notes' syntheses, your price targets and alerts, the symbol's fundamentals, recent news, and computed technicals (including the Confluence verdict), and asks an LLM — or a transparent rules engine if no key/quota — to return a clamped **action** (buy/watch/hold/sell), a **confidence**, a **rationale**, and bullet **factors**, which are stored with an **Assessed** timestamp (history capped at 3, changes logged, outcomes tracked). The **Recommendation** card simply re-presents that latest assessment and adds a **sentiment** chip that prefers a **market-grounded** read — how strongly and which way the stock actually reacted to its recent **News** (a beta-adjusted, volatility-standardized, volume-confirmed daily event study scored 0–100 and ranked by relevance × recency) — falling back to your note sentiment only when no materially-relevant news exists.